

Project 0

Advanced Compilers

Released: April 23, 2026

Due: May 10, 2026

In this project, you will implement heuristic optimization for a constant materialization routine on RISC-V 64-bit. In HW0, we inspected compiler-generated assembly, analyzed it, and observed the effect of applying optimization flags. This project builds on that experience with one concrete low-level task: given a target constant, emit a short instruction sequence that reconstructs each value in a simulator register. Data flow analysis-based optimization is an important part of compiler optimization, but a heuristic approach is still valuable here: it helps students get comfortable with assembly patterns first and develop practical optimization intuition before applying full compiler analyses.

1 Constant Materialization in RISC-V 64-bit

1.1 Why Multiple Instructions Are Needed

In RV64, each instruction is encoded in 32 bits. Those 32 bits must represent not only the operation itself, but also other required fields such as the opcode, register specifiers, and immediate bits. For that reason, a single instruction cannot directly encode and construct an arbitrary 64-bit constant.

As a result, when a compiler needs to place a 64-bit constant into a register, it must synthesize the value through multiple instructions. Each instruction contributes part of the final value, and the full constant is produced step by step.

1.2 Instructions for Constant Materialization

In this project, only five instructions are allowed in your implementation: `lui`, `addiw`, `addi`, `slli`, and `srli`. The simulator uses only one working register. The register is initialized to zero, and all emitted instructions are applied sequentially.

- `lui`

Uses a 20-bit immediate to construct bits 31:12 of the low 32-bit word and fills bits 11:0 with zeros. The resulting 32-bit value is then sign-extended to the full 64-bit working register.

Immediate constraint: the immediate used for `lui` must be representable as a signed 20-bit value.

Effect: $x \leftarrow \text{sext}(\text{imm20} \ll 12)$, where `sext` means sign-extension

- `addiw`

Adds a sign-extended 12-bit immediate to the current working register value. Any overflow beyond the low 32 bits is ignored. The low 32-bit result is then sign-extended back to the full 64-bit working register.

Immediate constraint: the immediate used for `addiw` must be representable as a signed 12-bit value.

Effect: $x \leftarrow \text{sext}((x + \text{sext}(\text{imm12}))[31 : 0])$

- **addi**

Adds a sign-extended 12-bit immediate to the current working register value. Arithmetic overflow is ignored, and the result is taken directly in the full 64-bit register width.

Immediate constraint: the immediate used for `addi` must be representable as a signed 12-bit value.

Effect: $x \leftarrow x + \text{sext}(\text{imm12})$

- **slli**

Performs a logical left shift on the current working register value. Bits shifted out on the left are discarded, and zeros are inserted into the low bits.

Immediate constraint: the shift amount used for `slli` must be representable as an unsigned 6-bit value in RV64.

Effect: $x \leftarrow x \ll \text{shamt6}$

- **srli**

Performs a logical right shift on the current working register value. Bits shifted out on the right are discarded, and zeros are inserted into the high bits.

Immediate constraint: the shift amount used for `srli` must be representable as an unsigned 6-bit value in RV64.

Effect: $x \leftarrow x \gg_u \text{shamt6}$

1.3 Baseline Implementation for Constant Materialization

One simple, correct, and arbitrary-constant-capable baseline is to construct the upper 32 bits first, and then append the remaining lower 32 bits step by step. In this project, the baseline implementation first uses `lui` and `addiw` to construct the upper 32-bit part. The use of `addiw` is important here because it discards overflow beyond 32 bits and sign-extends the resulting 32-bit value before the later 64-bit shifts are applied. It then uses repeated left shifts and `addi` to insert the remaining lower part.

One important restriction comes from `addi`. Since `addi` uses a sign-extended 12-bit immediate, using all 12 bits directly can unintentionally affect previously constructed higher bits. To avoid that problem in a simple baseline, the highest bit of the 12-bit immediate is fixed to zero, and only the lower 11 bits are used when appending lower chunks.

This baseline is already implemented in the given skeleton code. You should use that implementation as a reference when designing and testing your heuristic improvements.

With that idea, the baseline implementation in this project uses the following fixed sequence:

`lui, addiw, slli, addi, slli, addi, slli, addi`

with shift amounts 11, 11, and 10.

1.4 Heuristic Optimization for Selected Constants

Heuristic cases are used to produce shorter sequences for specific target constants.

Zero Constant

This case applies when the target constant is exactly zero. This case should still emit at least one instruction. You may choose a short valid shape, but it must stay within the allowed instruction set.

Signed 12-bit constant

This case applies when the target constant is representable as a signed 12-bit integer. Such a value can be handled with a very short immediate-based sequence.

Signed 32-bit Constant

This case applies when the target constant is representable as a signed 32-bit integer. You should emit a short sequence that produces the exact 64-bit target value.

Constant with at Least 32 Leading and Trailing Zeros in Total

This case applies when the total number of leading zeros and trailing zeros is at least 32. Such constants are structurally sparse, so a shorter sequence may be possible than the baseline. The exact split strategy is left to your implementation, but correctness is required.

2 Project Goal and Evaluation

2.1 Implementation Scope and Goal

In this project, you should implement only the TODO parts in the given source code, specifically inside `emitLoadConstant` in `load_constant.cpp`. Do not modify other source files. You should utilize methods in the `ProgramEmitter` class to materialize each target constant. For heuristic cases, the goal is to produce correct instruction sequences using fewer than 8 instructions whenever possible.

2.2 Constants Used for Evaluation

Only the following 10 constants will be used for evaluation. Your implementation should correctly materialize all of them:

- `0x0000'0000'0000'0000` (0)
- `0x0000'0000'0000'07FF` (2047)
- `0xFFFF'FFFF'FFFF'F800` (-2048)
- `0x0000'0000'0000'0123` (291)
- `0x0000'0000'7FFF'FFFF` (2147483647)
- `0xFFFF'FFFF'8000'0000` (-2147483648)

- 0x0000'0000'1234'5678 (305419896)
- 0x0000'0001'0000'0000 (4294967296)
- 0x0020'0000'0040'0000 (9007199258935296)
- 0x0000'0200'0000'0400 (2199023256576)

The provided program also prints one additional baseline demonstration constant in its built-in examples. That extra example is only for illustration and is not part of evaluation.

2.3 Test Your Implementation

1. Build your implementation:

```
$ make
```

2. Run the test:

```
$ ./load_constant
```

```
[START] constant 0xCAFE'BABE'DEAD'BEEF
case: fallback 8-instruction baseline
constant: 0xCAFE'BABE'DEAD'BEEF (-3819410105021120785)
instruction count: 8
```

Step	Instruction	Reg	Target	Match
1	lui 0xC'AFEC	0xFFFF'FFFF'CAFE'C000	0xCAFE'BABE'DEAD'BEEF	mismatch
2	addiw -1346 (0xABE)	0xFFFF'FFFF'CAFE'BABE	0xCAFE'BABE'DEAD'BEEF	mismatch
3	slli 11	0xFFFF'FE57'F5D5'F000	0xCAFE'BABE'DEAD'BEEF	mismatch
4	addi 1781 (0x6F5)	0xFFFF'FE57'F5D5'F6F5	0xCAFE'BABE'DEAD'BEEF	mismatch
5	slli 11	0xFFF2'BFAE'AFB7'A800	0xCAFE'BABE'DEAD'BEEF	mismatch
6	addi 879 (0x36F)	0xFFF2'BFAE'AFB7'AB6F	0xCAFE'BABE'DEAD'BEEF	mismatch
7	slli 10	0xCAFE'BABE'DEAD'BC00	0xCAFE'BABE'DEAD'BEEF	mismatch
8	addi 751 (0x2EF)	0xCAFE'BABE'DEAD'BEEF	0xCAFE'BABE'DEAD'BEEF	match

```
final: target=0xCAFE'BABE'DEAD'BEEF, result=0xCAFE'BABE'DEAD'BEEF, match
[END] constant 0xCAFE'BABE'DEAD'BEEF -> match
```

The provided driver first runs the built-in examples and then accepts user input. When `load_constant` runs, the given skeleton prints case labels and the register state at each step for debugging.

2.4 Implementation Tips

Many bugs in this project come from type interpretation rather than from the instruction sequence itself. Avoid implicit conversions between signed and unsigned 64-bit integers, and use `static_cast` explicitly whenever a signed immediate is intentionally derived from an unsigned bit pattern.

When manipulating bit patterns directly, prefer `uint64_t`. Use unsigned values for masking, chunk extraction, and logical shifts, and convert explicitly only when a signed immediate value is needed. In particular, avoid signed shift tricks for sign extension. It is usually clearer to extract a field first and then apply explicit sign extension, for example by using a helper such as `signExtendBits`.

It also helps to separate two questions while debugging: what bit pattern should be in the register after this step, and what immediate should be given to the next instruction to produce it? Use the printed table actively and find the first step where the register value diverges from the target. Also refer to the baseline fallback 8-instruction implementation.

2.5 Submission

Submit your `load_constant.cpp` as `PR0_<YOUR_STUDENT_ID>.cpp` (e.g., `PR0_2026-12345.cpp`) to eTL.

3 Reference

This document and source code comments are intended to be self-contained for this project. If you want additional details beyond what is included here, you can use:

- **official:** RISC-V Unprivileged ISA Specification.
https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf
- **unofficial:** A more readable RISC-V ISA documentation page with compact, instruction-focused explanations.
<https://msyksphinz-self.github.io/riscv-isadoc/>